

HW4 Answers – Korhan Erdoğan 30838

Q1 (20 pts)

Question: What are your newly added Turkish relationship predicates (there are 10 total)?

Answer:

The newly added Turkish relationship predicates are:

1. $\text{hala}(X,Y)$ – father's-side aunt (sister of Y's father)
2. $\text{teyze}(X,Y)$ – mother's-side aunt (sister of Y's mother)
3. $\text{dayi}(X,Y)$ – mother's-side uncle (brother of Y's mother)
4. $\text{amca}(X,Y)$ – father's-side uncle (brother of Y's father)
5. $\text{anneanne}(X,Y)$ – maternal grandmother (mother of Y's mother)
6. $\text{babaanne}(X,Y)$ – paternal grandmother (mother of Y's father)
7. $\text{gelin}(X,Y)$ – daughter-in-law (X is married to Y's son)
8. $\text{damat}(X,Y)$ – son-in-law (X is married to Y's daughter)
9. $\text{torun}(X,Y)$ – grandchild
10. $\text{dunur}(X,Y)$ – X and Y are *dünür* if their children are married to each other

Q2 (20 pts)

Question: Write a predicate `list-spouse-pairs` to list all spouses just once and without hitting next (e.g. similar to `woman_list` predicate we saw in slides). Include your query and the output of your program.

Answer:

I implemented the predicate `list_spouse_pairs/0`, which prints each married couple **only once** by requiring `X @< Y` (alphabetical order). It prints each pair on a separate line, then forces backtracking with `fail`, and finally succeeds.

Query:

```
?- list_spouse_pairs.
```

Output (one possible order):

ali - ayse

remzi - rukiye

mualla - osman

murat - oya

true.

(Note: order of lines may vary depending on solution order, but it always prints each couple exactly once due to `X @< Y`.)

Q3 (20 pts)

Question: Is there someone who has at least two children, who are married (i.e., non-single)? Include your query and the output of your program.

Answer:

Yes. The following query searches for a person P who has two different children C1 and C2, and both children appear in spouse(_,Child) (meaning they are married / not single). C1 @< C2 avoids duplicate child-pair ordering.

Query:

```
?- person(P),  
    child(C1,P), spouse(_,C1),  
    child(C2,P), spouse(_,C2),  
    C1 @< C2.
```

Expected output for this database:

P = ayse,

C1 = osman,

C2 = oya ;

P = ali,

C1 = osman,

C2 = oya ;

P = rukiye,

C1 = mualla,

C2 = murat ;

P = remzi,

C1 = mualla,

C2 = murat ;

false.

Q4 (10 pts)

Question: Is the order of the clauses of the add-person predicate matter? (e.g. Can I move add(...) as the last line of the predicate?)

Answer as YES/NO and add a one-line explanation.

Answer:

YES. Prolog runs goals top-to-bottom, and add_person/5 must call add(...) before the integrity checks; also the rollback behavior depends on the current structure (the “rollback on backtracking” add/5 clause).

Q5 (30 pts)

Question: What is done in `mother_check`. Explain as: “it first checks
If this succeeds/fails,”

Answer:

`mother_check` performs semantic integrity checks related to a person's mother in this order:

1. It first checks whether the mother exists in the database.

```
mother_check(_, _, Mother) :- \+(person(Mother)), !.
```

If Mother is **not** a known person/5 fact, it **succeeds immediately** and **cuts** (!), meaning: “we can't validate further, so accept it.”

2. If the mother exists, it checks whether the mother is male.

```
mother_check(_, _, Mother) :-
```

```
    male(Mother),
```

```
    assert(message("Person's mother is a man")),
```

```
    !, fail.
```

If the mother is found and `male(Mother)` succeeds, it asserts an error message and then **fails** (with a cut) to reject the update.

3. Then it checks a different kind of inconsistency: whether the newly added person is male but appears as someone's mother.

```
mother_check(Name, male, _) :-
```

```
    mother(Name, _),
```

```
    assert(message("Person, a male, is someone's mother")),
```

```
    !, fail.
```

If the person being added is male and `mother(Name, _)` succeeds (meaning the database would treat this male as a mother of someone), it asserts a message and **fails** (cut + fail).

4. Otherwise it succeeds.

```
mother_check(_, _, _).
```

If none of the above error cases apply, the predicate succeeds normally.

Korhan Erdoğan 30838